



Null

🕒 Created time	July 7, 2026 4:35 AM
☰ Category	CommonVariables
👤 Last edited by	 R.A.R.A
🕒 Last updated time	July 7, 2026 5:28 AM

```
//Null - Lack of values.  
  
//Haven't asked for the age yet  
int? age = null;  
bool? birthday = null;  
double? battingAverage = null;  
decimal? accountBalance = null;  
string? firstName = null;  
  
//we have asked for the age now  
age = 0;
```

ה-Null הוא חוסר בערכים.

ערך **Null** יכול להיות מוקצה לכל סוג משתנה אם נגדיר שזה אפשרי.

נדמין את זה כחור שחור של ערכים.

הסיבה שיש לנו **Null** היא כדי לציין שאין כאן כלום.

לדוגמה, יש לנו משתנה גיל, ערך ברירת המחדל שלו הוא **0**.

מה אם עדיין לא שאלנו את האדם לגילו?

האם עלינו לשים שם **0** כנראה שלא, כי מבחינה טכנית **0** הוא גיל.

אנחנו עשויים להגיד, לא! **0** הוא לא גיל! אבל כשתינוק רק נולד, מה הגיל שלו בשנים? מכיוון שהם מגיעים ליום ההולדת הראשון שלהם **365** ימים אחרי שנולדו, אז מה הגיל שלהם בשנים עד לאותה נקודה? ובכן... כנראה **0**.

בארה"ב אומרים דברים כמו "הם בני שבוע", "הם בני חודשיים", כן אבל הם עדיין לא בני שנה ואנחנו משתמשים כאן במספרים שלמים עבור שנים כי ככה אנחנו מדברים.

אנחנו לא אומרים "אני בן **3.45**", באופן כללי אנחנו לא עושים את זה, לא אומרים **3.5**, ולא אומרים "אני בן **43.25**".

אנחנו אומרים **43** או שאנחנו אומרים **3**, ולכן אנחנו משתמשים בשפה היומיומית שלנו במספרים שלמים כדי לציין את השנה שלנו.

אנחנו לא קוראים להם מספרים שלמים כי אנחנו בכלל לא יודעים שזה מספר שלם, אנחנו אומרים מספרים שלמים או פשוט נותנים מספר.

אז אם גיל יכול להיות 0 ועדיין לא שאלנו את המשתמש לגילו, האם זה יכול לבלבל? כי ממש עכשיו, ממש כאן, עדיין לא שאלנו לגיל:

```
age = 0;
```

ואם זה תינוק ועכשיו שאלנו לגיל:

```
age = 0;
```

וההבדל הוא כלום.

אז איך אנחנו אומרים למערכת אם זה גיל תקין או גיל שעדיין לא שאלנו עליו? ובכן, כאן **Null** נכנס לתמונה, כי **Null** מאפשר לנו להגיד "חסר לזה ערך, עדיין אין לזה ערך". בדרך כלל נחשוב להגיד:

```
int age = null;
```

אבל גיל אינו יכול להכיל **Null**.

כלומר, אין לו את היכולת לאחסן חוסר בערך.

אז אנחנו משתמשים בסימן השאלה הזה בסוף סוג המשתנה:

```
int? age = null;
```

סימן השאלה מציין שהסוג הזה הוא מסוג **Nullable**, כלומר יש לו את היכולת לאחסן **Null**.

אז עכשיו הגיל עדיין לא הוגדר:

```
int? age = null;
```

ועכשיו הגיל הוגדר וזה הערך:

```
age = 0;
```

זה ממש שימושי עבור נתונים, כי במסד הנתונים יש לנו לעתים קרובות את המושג של **Null**.

אם אין לנו ערך לעמודה, אנחנו לא שמים שם ערך, אנחנו אומרים שזה **Nullable** ואומרים שערך העמודה הזו עדיין לא קיים ולכן הוא **Null**.

אז כשזה מגיע להכנסת הנתונים האלה ל-**C#**, אנחנו לא רוצים להמיר את זה מ-**age = 0** לערך ברירת מחדל שאינו **Null**, כי אז נדרוס את ערך ה-**Null** עם ערך שנראה אמיתי.

אז אנחנו רוצים להיות מסוגלים לשמור על ה-**Null** הזה הלוך ושוב, ולכן אנחנו משתמשים בתווים המאפשרים **Null**. הדבר הנחמד כאן הוא שיש לנו את האפשרות הזו:

```
int? age = null;
```

כי אם נוריד את סימן השאלה הזה, גיל לא יכול להיות **Null**, והוא אינו אפשרות.

אנחנו יכולים לעשות את זה עם כל הסוגים, כמו עם `int?`, `bool?`, `double?`.

לכן, המשתנה

```
double? battingAverage = null;
```

הוא `battingAverage` היכול להכיל `Null` כי אנחנו עדיין לא יודעים מה הממוצע, ולכן לא נשים כאן מספר כמו `0` כשהקומפילר יגיד "רגע, `battingAverage` שלך הוא `0`? זה לא נהדר!".

לא! כי `0` יכול להיות מספר תקין, אנחנו נגיד `Null` עד שתיתנו לנו מספר תקין, שיכול להיות `0`.

וזה נכון לכל הסוגים, אנחנו יכולים לשים שם את סימן השאלה שמציין שזה `Nullable`.

בפיתוח מערכות תוכנה מתקדמות, אנחנו חייבים דיוק מוחלט בניהול המידע שלנו.

בשפת `#C`, סוגי משתנים פרימיטיביים כמו מספרים או משתנים בוליאניים חייבים להחזיק בערך כלשהו מהרגע שיוצרים אותם.

אם ניקח את המשתנה `int` המייצג מספר שלם, ערך ברירת המחדל שלו הוא תמיד `0`.

האתגר האמיתי מתחיל כשאנחנו צריכים להבדיל בין מצב שבו המשתמש באמת הזין את המספר `0`, לבין מצב שבו הוא פשוט עדיין לא הזין שום נתון (`Null`).

כדי לפתור את חוסר הוודאות הזה בצורה חכמה ובטוחה, אנחנו מוסיפים את סימן השאלה (?) לאחר סוג המשתנה. הפעולה הפשוטה הזו משנה את ההתנהגות של המשתנה מאחורי הקלעים והופכת אותו למה שנקרא **Nullable Type**. המשמעות היא שהמשתנה מקבל יכולת חדשה: הוא יכול עכשיו להחזיק "כלום" מוחלט, שמיוצג על ידי המילה `null`.

הגישה הזו קריטית מאוד היום בעבודה מול בסיסי נתונים וממשקי **API** ברמת תעשייה.

כשאנחנו שולפים נתונים ולא קיים ערך במסד הנתונים, אנחנו רוצים שגם בקוד שלנו המשתנה יהיה ריק לחלוטין ולא יקבל בטעות את הערך `0` שעלול להרוס לנו את הלוגיקה של האפליקציה או להציג מידע שגוי ללקוח הקצה.

השימוש ב-**Nullable Types** מאפשר לנו לכתוב קוד נקי, מודרני, וחסין מפני שגיאות של נתונים חסרים.

~~~~~

```
string firstName = null;  
string? firstName = null;  
<Nullable>enable</Nullable>
```

## מחרוזת (String) היא קצת שונה.

היא לא `Nullable`.

זה טכנית עובד:

```
string firstName = null;
```

הקומפיילר מאפשר את זה כי מחרוזת היא סוג שונה, זה מה שנקרא בעצם **Struct**, אבל מה שזה אומר עבורנו זו העובדה שכברירת מחדל אנחנו יכולים לשים **Nulls** בתוך מחרוזות בלי לסמן אותן כ-**Nullable**.

עם זאת, יש פיצ'ר חדש ב-**#C** שאומר שכנראה כדאי לנו לעשות בדיקה על זה, ועשוי לפחות לזרוק אזהרה גדולה שאומרת שזה לא דבר כל כך טוב.

מכיוון שבמקרה זה אנחנו לא אומרים שאנחנו מצפים לערך **Null** כאן כי זו מחרוזת, זו לא מחרוזת **Nullable**, אז למרות שטכנית היא יכולה לאחסן **Null**, אנחנו לא אומרים שאנחנו מצפים ל-**Null**.

לכן **Visual Studio** אומר שזה עשוי להיות באג, ואנחנו מבינים שזה עשוי להיות ו-**Visual Studio** צודק.

אנחנו מבינים שאנחנו מכניסים **Null** לתוך ערך שאנחנו אומרים שהוא נדרש ולכן זו בעיה, וזה יכול לשנות את הקוד שלנו, או שאנחנו יכולים לשים סימן שאלה כזה:

```
string? firstName = null;
```

ולהגיד ל-**Visual Studio** שאנחנו מצפים שזה יהיה **Null** בשלב מסוים או יכול להיות **Null** על ידי שימוש בתו סימן השאלה, ועכשיו **Visual Studio** כבר לא יצעק עלינו.

אז זה לא יאלץ אותנו להפוך מחרוזות ל-**Nullable** כי הן כבר כאלה, אבל זה יזהיר אותנו אם ננסה להשתמש בערך **Null**, בגלל העובדה שלא אמרנו שאנחנו מצפים ל-**Null**.

אז זה סוג של מוזרות קטנה בעניין הזה, אבל עם זה: **string?**, זה מאפשר לקוד שלנו להיות קצת יותר מדויק וזה סוג של עוזר לנו ב-**Visual Studio** בכך שזה אומר לנו שזה עשוי להיות בעיה, ואם כן אז נוכל להגיד שאו שלא נהפוך את זה ל-**Null** או שנהפוך את המחרוזת הזו ל-**Nullable**.

אז האפשרויות האלה מאפשרות לנו להגיד מה קורה ו-**Visual Studio** אז עוזר לנו להבטיח שאנחנו עוקבים אחר הרצונות שלנו.

האזהרה הזו היא לא פיצ'ר חדש של **Visual Studio**, זה בגלל שיש לנו הגדרת **Nullable** בפרויקט שלנו, בקובץ הפרויקט תחת "**Edit project file**" יש לנו את השורה:

```
<Nullable>enable</Nullable>
```

אנחנו יכולים למחוק את זה ולא נקבל יותר את האזהרות האלה, אבל כדאי לנו להשאיר אותן כדי שיעזרו לנו.

## מנגנון שנועד להגן עלינו מקריסות קוד

היסטורית, מחרוזת ב-**#C** תמיד יכלה לקבל ערך ריק (**Null**).

**חשוב לדייק כאן:** ש-**String** זה **Struct**, אבל למעשה **String** הוא **Reference Type** (טיפוס הפנייה), וזו בדיוק הסיבה שהוא תמיד יכל להכיל **Null**, כי הוא מצביע למיקום בזיכרון, ואפשר להצביע על "כלום".

הבעיה היא שכאשר יש לנו משתנה שמכיל **Null** ואנחנו מנסים לעשות עליו פעולות בהמשך הקוד, האפליקציה שלנו עלולה לקרוס עם שגיאה מפורסמת שנקראת "**Null Reference Exception**".

כדי לפתור את זה, החל מגרסאות מתקדמות של **#C**, מייקרוסופט הוסיפו הגנה לפרויקטים שמופעלת כברירת מחדל דרך השורה בקובץ ההגדרות.

**ההגנה הזו משנה את חוקי המשחק:** היא אומרת שמעכשיו, אם הגדרנו **String** רגיל, הקומפיילר מניח שחייבים לשים שם ערך אמיתי.

אם בכל זאת ננסה לשים שם **Null**, ה-**Visual Studio** לא יעצור את התוכנית, אבל הוא יסמן קו אזהרה (**Warning**) כדי להתריע שיש פה מוקש פוטנציאלי שעלול לגרום לבאג.

הפתרון המקצועי לזה הוא להשתמש בסימן השאלה (?) ולהגדיר את המשתנה בתור **string?**.

ברגע שנוסיף את סימן השאלה (?), בעצם נצהיר מול הקומפיילר שאנו יודעים מה אנו עושים ושהמשתנה הזה מיועד להכיל **Null** כחלק מהלוגיקה שלנו.

זה משתיק את האזהרה, עושה סדר בכוונות שלנו, והופך את הקוד להרבה יותר בטוח, קריא וברור.